

[Your University] [School / Faculty] [Department of Computer Science]

SVDS — VEHICLE REGISTRY AND STOLEN VEHICLE DETECTION SYSTEM

**A camera-based license plate detection and vehicle registry system for the Zambia
Police Service**

BY

[Your Full Name]

[Course Code] FINAL YEAR COURSE PROJECT

A THESIS SUBMITTED IN PARTIAL FULFILMENT OF THE REQUIREMENT FOR THE
AWARD OF A BACHELOR'S DEGREE OF COMPUTER SCIENCE

SUPERVISOR: [Supervisor Name]

DECLARATION

I, the undersigned, hereby declare that the Vehicle Registry and Reporting System / Stolen Vehicle Detection System (VRRS/SVDS) is my own work. That it has not been submitted for any degree or examination in any other university to my knowledge, and that all sources I have used or quoted have been indicated and acknowledged by complete references.

Author: **[Your Full Name]** Student Number: **[Student Number]** Signature: _____ Date:
[Submission Date]

Supervisor: **[Supervisor Name]** Signature: _____

DEDICATION

[Write your own — the golf thesis dedicated the work to family/faith; keep this section personal to you rather than templated.]

ACKNOWLEDGEMENT

I want to express my gratitude to [Supervisor Name], my supervisor, for guidance and feedback throughout this project. [Add anyone else who supported the work — family, the Zambia Police Service contacts who informed the requirements interviews, etc.]

ABSTRACT

Vehicle theft investigation in Zambia currently relies on manual, paper-based reporting and word-of-mouth alerting between police stations — a stolen vehicle report filed at one station has no mechanism to automatically flag the vehicle if it is later sighted elsewhere. This project, SVDS (Stolen Vehicle Detection System), addresses that gap by combining a conventional vehicle-registry web application with a real-time, camera-based automatic license plate recognition (ALPR) pipeline.

The system has three cooperating parts. A FastAPI backend exposes a REST and WebSocket API over a PostgreSQL database, enforcing role-based access for three user types: Reportee (the public, filing and tracking stolen-vehicle reports), Police (viewing live detection alerts, managing the vehicle registry, and analytics), and Admin (user and system management). A React single-page application provides the interface for all three roles. A separate, decoupled camera-node process streams video from a phone-mounted camera, detects license plates with a custom-trained YOLOv8n object detector, reads the plate text with EasyOCR, and reports matches to the backend over plain HTTP — requiring no coupling to the backend's internals or technology choices.

The detection model was trained on a self-collected, Roboflow-annotated dataset of Zambian number plates (335 source images, augmented to 1,890) and evaluated at precision 0.988 and recall 0.751 (mAP50 0.801) after 100 training epochs. The full system was tested end-to-end — from a real phone camera feed, through detection and OCR, to a live WebSocket alert appearing on the police dashboard — and validated with an automated test suite (39 tests) covering authentication, role-based access control, the report lifecycle, plate-matching logic, and the analytics layer.

This work demonstrates that a real-time ALPR pipeline can be integrated into a practical, role-based law-enforcement registry system using consumer hardware (an Android phone as the camera) and open-source computer vision tooling, without recourse to commercial ALPR licensing.

1. Introduction

SDLC phase: Problem identification / feasibility study

1.1 Background

Vehicle theft is a persistent problem for law enforcement in Zambia, and the process of recovering a stolen vehicle depends heavily on chance: an officer at a checkpoint or on patrol happening to recognize a plate number from a paper report or a radio bulletin. Vehicle registry records, incident reports, and eyewitness descriptions are typically kept on paper or in disconnected station-level spreadsheets, which means a report filed at one police station is invisible to an officer working in a different part of the same city, let alone a different province.

At the same time, automatic license plate recognition (ALPR) has become inexpensive enough to build from commodity hardware. A single Android phone can stream video over Wi-Fi, and open-source object detection models (this project uses Ultralytics YOLOv8) can be trained on a modest, self-collected image set and run in real time on a consumer GPU. This closes the gap between "a report exists somewhere" and "a report is automatically checked against every vehicle a camera sees."

1.2 Problem Statement

There is no system available to the Zambia Police Service that automatically checks a vehicle's license plate — as seen by a camera in the field — against an active stolen-vehicle registry in real time. Existing practice depends on manual recognition by officers and manual cross-referencing of paper reports. This results in: (a) slow or missed detections of stolen vehicles that pass through monitored locations, (b) no centralized, real-time way for a report filed by the public to reach every officer who might encounter the vehicle, and (c) no audit trail connecting a detection event back to the report that triggered it.

1.3 Research Questions

1. Can a custom-trained, lightweight object detector (YOLOv8n) combined with an open-source OCR engine achieve detection and recognition accuracy sufficient for real-time plate matching on Zambian number plates, using only a phone camera and a consumer GPU?

2. Can such a detection pipeline be integrated into a role-based registry system as a fully decoupled component — communicating only over plain HTTP/WebSocket — without requiring the detection logic to be embedded in the backend itself?
3. Does a real-time alert delivered over WebSocket meaningfully reduce the latency between "a stolen vehicle is sighted" and "a police officer is notified," compared to the manual process it replaces?

1.4 Aim

The aim of this project is to design, implement, and evaluate SVDS: a vehicle registry and reporting web application integrated with a real-time, camera-based license plate detection pipeline, for use by the Zambia Police Service and the public it serves.

1.5 Objectives

- Implement role-based accounts for three user types — Reportee, Police, and Admin — each with access scoped to what that role needs (`vrrs-backend/app/middleware.py`).
- Allow the public to file, track, and view the status of stolen-vehicle reports (`under_review` → `missing` → `found`).
- Give police officers a live view of detection alerts, the vehicle registry, and system analytics, with the ability to activate reports, mark vehicles recovered, and flag false positives.
- Give administrators control over user accounts, roles, and visibility into system health and the audit trail.
- Collect and annotate a dataset of Zambian license plates, and train a YOLOv8-based object detector to localize plates in a camera frame.
- Integrate an OCR stage (EasyOCR) to read the localized plate text, and validate/clean that text against the expected format.
- Build a decoupled camera-node process that connects a real video stream (an Android phone's IP Webcam feed) to the detection pipeline and reports matches to the backend over HTTP, independent of the backend's and frontend's technology stack.
- Deliver real-time alerts to connected police/admin clients over WebSocket the moment a stolen vehicle is detected.
- Evaluate the detector quantitatively (precision, recall, mAP) and the system as a whole through automated and manual/system testing.

1.6 Scope

This project covers the vehicle-registry web application (backend + frontend), the camera-node detection pipeline, and the integration between them. It does **not** cover: a native mobile app for the public (the public-facing interface is a responsive web app), payment/e-government integration, multi-camera fleet management beyond a single configured node, or GPS-based camera geolocation (the camera's location is currently a configured string, not read from the phone's sensors — see §9.2).

2. Literature Review

SDLC phase: Research / analysis (precedes formal requirements gathering)

2.1 Introduction

This chapter surveys existing automatic license plate recognition (ALPR) approaches and law-enforcement/vehicle-registry systems relevant to SVDS, to establish what already exists, what is transferable, and what gap this project fills — specifically, the absence of any ALPR-integrated stolen-vehicle registry built for the Zambian context.

2.2 Review of Similar Systems

2.2.1 Commercial and open ALPR platforms

- **OpenALPR** — an open-source ALPR library offering pre-trained models for several countries' plate formats. It is a strong general-purpose reference but ships no model trained on Zambian plates, and its default pipeline (classical image processing plus a small CNN) predates the anchor-free, single-stage detectors (e.g., YOLOv8) used in this project.
- **Plate Recognizer** (a commercial ALPR SaaS) — demonstrates the split this project also uses: a lightweight edge client that captures frames and a separate service that performs recognition. Its accuracy is strong across many plate formats but the service is a paid API, which is a barrier for public-sector deployment on a limited budget — reinforcing the case for a self-hosted, custom-trained alternative.
- **Ultralytics YOLOv8** (the detector family used in this project) — chosen over older YOLO versions and over two-stage detectors (e.g., Faster R-CNN) for its balance of inference speed and accuracy on a single consumer GPU, and its first-class support for training on a small, custom, single-class dataset via the Ultralytics CLI/Python API.
- **EasyOCR** — an open-source, PyTorch-based OCR engine supporting a character allowlist, which is used here to constrain readings to the alphanumeric character set a Zambian plate can contain, reducing spurious reads from background text.

2.2.2 Vehicle registry and reporting systems

Existing vehicle registry and reporting workflows in the literature and in observed practice are predominantly record-keeping systems: they store reports, ownership records, and status, but do not themselves watch for the reported vehicle. Where ALPR is used in policing literature, it

is typically deployed as fixed, purpose-built roadside camera infrastructure (e.g., number-plate recognition cameras on major routes) rather than integrated with a public-facing reporting system that a citizen can use directly. SVDS's contribution is closing that loop: a report filed by the public immediately becomes something a camera node can match against, without a separate data-entry step for police to "load" the report into a detection system.

2.2.3 Gap this project addresses

No system reviewed combines all of: (a) public self-service reporting, (b) role-based access for police/admin oversight, (c) a custom-trained detector for the specific plate format in question, and (d) real-time WebSocket delivery of detection alerts to connected officers. SVDS's architecture — a fully decoupled camera node talking to a conventional registry backend over HTTP — is deliberately chosen so the detection pipeline could later be pointed at fixed roadside cameras without changing the backend at all.

2.3 Research Methodology

Requirements and design decisions for SVDS were informed by:

- **Background reading** on ALPR architectures (YOLO family detectors, OCR engines) and on typical vehicle-registry data models (report lifecycle, audit logging).
- **Direct inspection of the problem domain** — the manual, paper-based nature of current stolen-vehicle reporting, and the fact that police, reporters, and administrators need materially different views of the same underlying data (which directly shaped the three-role design).
- **Iterative build-and-test cycles**, where the camera-node pipeline (frame capture → detection → OCR → cleaning → dedup → HTTP report) was refined against real footage from a phone camera rather than only against the training/validation image set.

2.4 Summary

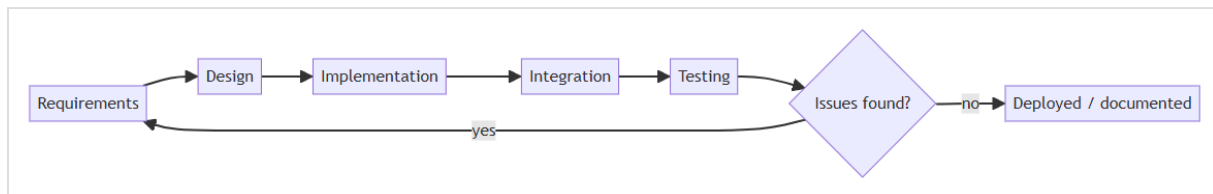
ALPR technology and vehicle-registry web applications are both mature, well-documented domains individually. The literature and available products show little precedent for combining the two into a single system aimed at a specific national plate format and a public/police/admin role split — which is the space this project occupies.

3. Requirements Engineering

SDLC phase: Requirements

3.1 Development Methodology

SVDS was built iteratively rather than to a single upfront specification: the registry web application (backend + frontend) and the camera-node detection pipeline were developed as two work-streams that converged at one integration point — the `POST /alerts/check-plate` endpoint. This let the detection model's training/evaluation cycle (data collection → annotation → training → evaluation, see Chapter 5) proceed independently of the web application's CRUD/auth work, and let each be iterated and retested without blocking the other.



3.2 Required Resources

Layer	Technology
Backend	Python, FastAPI, SQLAlchemy (ORM), PostgreSQL, JWT (<code>python-jose</code>), <code>bcrypt (passlib)</code>
Frontend	React 18, React Router, Axios, Vite
Camera node	Python, OpenCV, Ultralytics YOLOv8, EasyOCR, PyTorch (CUDA)
Dataset/annotation	Roboflow (annotation, augmentation, dataset versioning/export)
Dev/runtime	Windows 11, NVIDIA GPU (CUDA) for training and real-time inference, Android phone running an IP Webcam app as the camera source

Development environments are isolated per component: the backend uses a Python venv with `psycopg2-binary` for Postgres connectivity; the camera node requires a GPU-enabled Python environment with `torch / ultralytics / easyocr` installed (CPU-only inference is possible but materially slower for real-time video).

3.3 Requirements Gathering Approach

Requirements were derived from: (a) the practical gap identified in Chapter 1 (no automatic cross-referencing of reports against field sightings), (b) the natural three-way split of who interacts with a stolen-vehicle report — the person who filed it, the officer investigating it, and the administrator managing accounts and system health — and (c) the technical constraints of the detection pipeline itself (a single, cheap camera source; the need to avoid coupling the detector to the backend's release cycle).

3.4 Functional Requirements

3.4.1 Reportee (public)

- Register an account and log in (`POST /auth/register` , `/auth/login`).
- File a stolen-vehicle report with owner, vehicle, and incident details (`POST /reports/`); a duplicate report cannot be filed against a plate that already has an active (`missing` -status) report.
- View and search their own filed reports (`GET /reports/` , scoped to their own `reported_by`).
- View and edit their own profile (`GET/PATCH /users/me`), including changing password.

3.4.2 Police

- Everything a Reportee can do, plus:
- View, search, and filter all reports system-wide, not just their own.
- Activate a report (move it from `under_review` to `missing` , making it eligible for plate matching) and mark a report `found` once the vehicle is recovered (`PATCH /reports/{id}/activate` , `PATCH /reports/{id}/found`).
- View live detection alerts, mark them read, and flag false positives (`GET /alerts/` , `PATCH /alerts/{id}/read` , `PATCH /alerts/{id}/false-positive`).
- Receive new detection alerts in real time via WebSocket without refreshing the page.
- View the live camera feed and camera-node status (online/offline, detection count, average confidence) on the Cameras page.
- View system analytics: report totals/status breakdown, recovery rate, alerts over time, detections per camera node, average recovery time.

3.4.3 Admin

- Everything a Police user can do, plus:
- Create police/employee accounts (`POST /users/invite`), change any user's role (`PATCH /users/{id}/role`), deactivate/reactivate accounts, and delete accounts.

- Delete a stolen-vehicle report, including one with existing detection history (`DELETE /reports/{id}` — see §8.5 for why this specifically needed a fix).
- View the system-wide audit log of significant actions (`GET /system/audit`).
- View system health: API response time, DB connection pool usage, disk usage, active sessions, camera node online/offline count, uptime (`GET /system/health`).
- View user-growth analytics (`GET /analytics/user-growth`).

3.4.4 Camera node (SVDS detection subsystem)

- Connect to a live MJPEG video stream from a phone-mounted camera.
- Detect license plates in each processed frame using a custom-trained object detector, at a configurable confidence threshold.
- Extract and pre-process the detected plate region for OCR (grayscale, upscale, threshold).
- Read the plate text and validate it against the expected Zambian plate character pattern before treating it as a genuine reading.
- Suppress repeated reports of the same plate within a cooldown window, tolerant of OCR noise between frames (i.e., near-identical readings should not each trigger a separate report).
- Report each accepted reading to the backend over HTTP, and log the backend's response (CLEAR/STOLEN) locally.
- Recover automatically if the video stream connection drops.

3.5 Non-Functional Requirements

- **Security:** passwords are never stored in plaintext (bcrypt hashing); every protected route requires a valid JWT; role checks are enforced server-side (`app/middleware.py` 's `require_role`), not only hidden in the frontend UI; the camera feed proxy and WebSocket endpoint both validate a JWT before granting access, since `<img>` tags and WebSocket handshakes cannot send an `Authorization` header.
- **Performance:** the detection pipeline processes every 5th frame (not every frame) to keep pace with the video stream on the available GPU; database aggregate queries in the analytics/system routers use SQL-side counting/averaging rather than pulling full row sets into Python.
- **Availability/resilience:** the camera node reconnects automatically if the phone stream drops; the backend's WebSocket manager removes dead connections on broadcast failure rather than blocking on them.
- **Decoupling:** the camera node has no dependency on the backend's internals — it is a separate OS process communicating only over HTTP/JSON, so it can be redeployed,

restarted, or replaced independently (see Chapter 7).

- **Usability:** each role sees only the navigation and actions relevant to it (the frontend's route guards mirror the backend's role checks) so users are not shown controls they cannot use.
 - **Maintainability:** the schema is defined once in SQLAlchemy models and shared by every router; business rules that matter to more than one endpoint (e.g., confidence normalization) are factored into a single expression rather than duplicated.
 - **Auditability:** state-changing actions taken by police/admin users are written to an `audit_log` table, giving a traceable record of who activated, resolved, or deleted a report, and who changed a user's role.
-

4. System Design

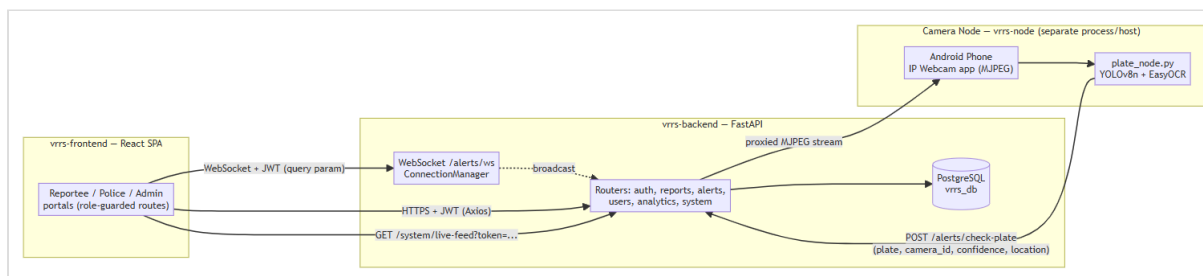
SDLC phase: Design

4.1 Proposed System

SVDS is composed of three independently-runnable processes that communicate only over network calls, never by sharing code or a process boundary:

1. **vrrs-backend** — a FastAPI application backed by PostgreSQL, exposing REST endpoints and one WebSocket endpoint.
2. **vrrs-frontend** — a React single-page application consumed by browsers, talking to the backend over HTTPS (Axios) and WebSocket.
3. **vrrs-node** — a standalone Python script (`plate_node.py`) that pulls a phone's video stream, runs detection/OCR, and reports over plain HTTP.

4.2 System Architecture



This is deliberately not a monolith: the camera node does not import anything from **vrrs-backend**, and could be pointed at a different backend entirely by changing one environment variable (`BACKEND_URL`). This decision trades a small amount of network overhead for the ability to develop, restart, and scale the detection pipeline independently of the web application — see Chapter 7 for the integration contract in detail.

4.3 Database Design (Entity-Relationship Diagram)

The schema has four tables, defined once in `vrrs-backend/app/models.py` and shared by every router:

USERS			
int	id	PK	
string	first_name		
string	last_name		
string	email	UK	
string	phone		
string	password_hash		
string	role		reportee police admin
string	national_id		
string	badge_number		
bool	is_active		
datetime	created_at		

files (reported_by)

REPORTS			
int	id	PK	
int	reported_by	FK	
string	owner_name		
string	license_plate		
string	vehicle_make		
string	vehicle_model		
string	vehicle_color		
string	vehicle_year		
string	chassis_number		
datetime	incident_date		
string	status		under_review missing found
text	last_seen_location		
text	description		
datetime	report_date		
datetime	resolved_date		
int	resolved_by	FK	

triggers (report_id)

ALERTS			
int	id	PK	
int	report_id	FK	
text	location_spotted		
string	camera_id		

performs (performed_by)

AUDIT_LOG			
int	id	PK	
int	performed_by	FK	
string	action		
string	target_table		
int	target_id		
datetime	timestamp		

float	confidence_score	
string	image_path	
bool	is_read	
datetime	detected_at	

`Alert.report_id` carries `ON DELETE CASCADE` (see §8.5): deleting a report also deletes its detection history rather than being blocked by the foreign key. `Report` has no direct foreign key to `Alert`; the relationship is navigated the other way, and `Alert.license_plate` is a computed property that reads through to `Alert.report.license_plate` rather than being stored redundantly.

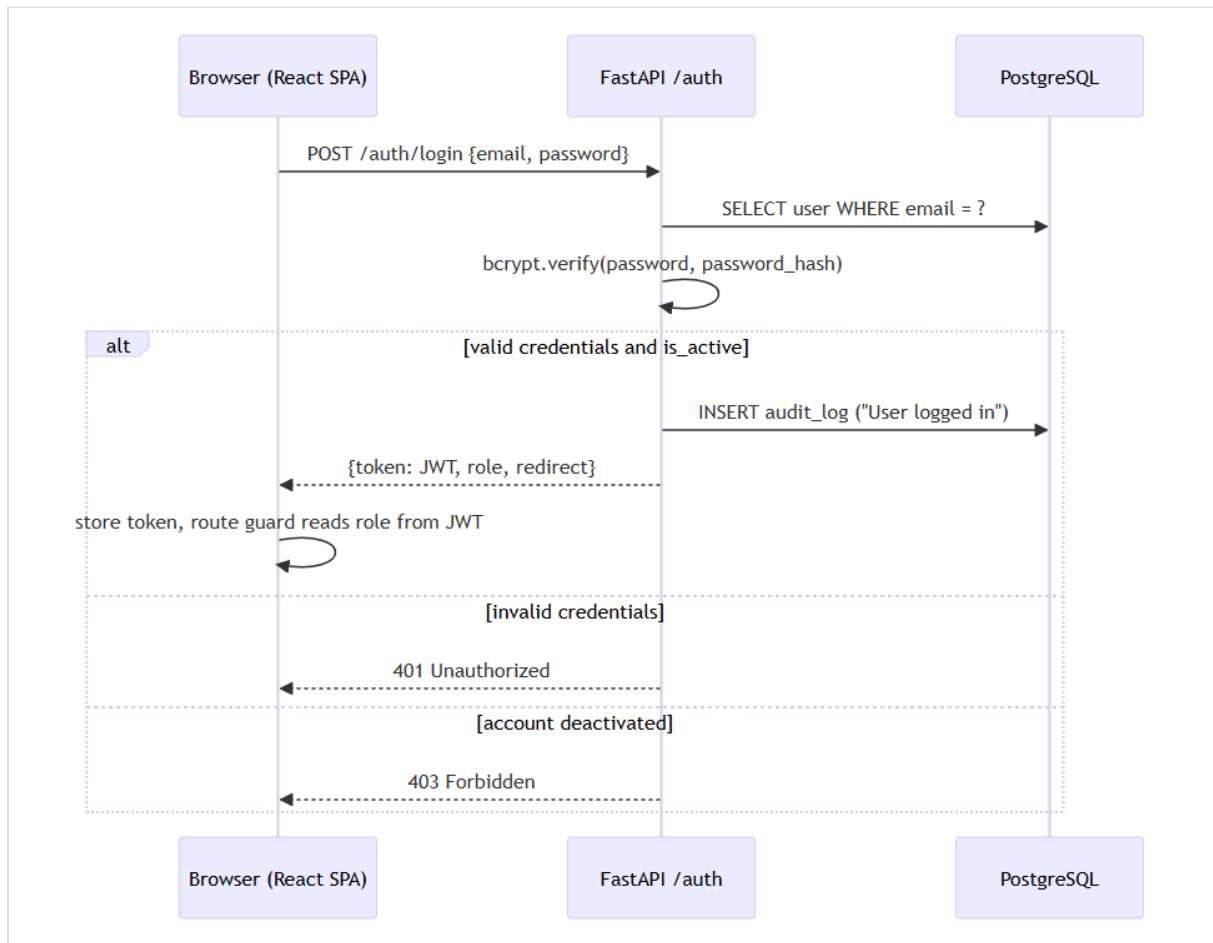
No formal migration tool (e.g., Alembic, though it is listed in `requirements.txt`) is currently wired up — the schema is created with `Base.metadata.create_all()` at backend startup, which creates missing tables but does not alter existing ones. This is why the cascade fix in §8.5 required a direct `ALTER TABLE` against the live database in addition to the model change (see §9.2 for this as a noted limitation).

4.4 API Design

Router	Prefix	Purpose	Auth
Auth	<code>/auth</code>	Register (reportee only), login, logout	Public
Reports	<code>/reports</code>	Stolen-vehicle report CRUD + lifecycle transitions	Reportee+ (scoped), Police for updates, Admin for delete
Alerts	<code>/alerts</code>	<code>check-plate</code> (public, called by the camera node), alert listing/read/false-positive, WebSocket	Public for <code>check-plate</code> ; Police+ for the rest
Users	<code>/users</code>	Profile, admin user management, role changes	Self for profile; Admin for management
Analytics	<code>/analytics</code>	Aggregate stats for dashboards/charts	Police+ (Admin for user-growth)
System	<code>/system</code>	Audit log, camera-node status, live-feed proxy, health metrics	Admin/Police, token-gated for the feed proxy

`/alerts/check-plate` is intentionally unauthenticated (the camera node has no user identity of its own) but is not a general write endpoint — it can only ever produce a `CLEAR` response or create an `Alert` row linked to an already-missing report; it cannot create, modify, or delete a report.

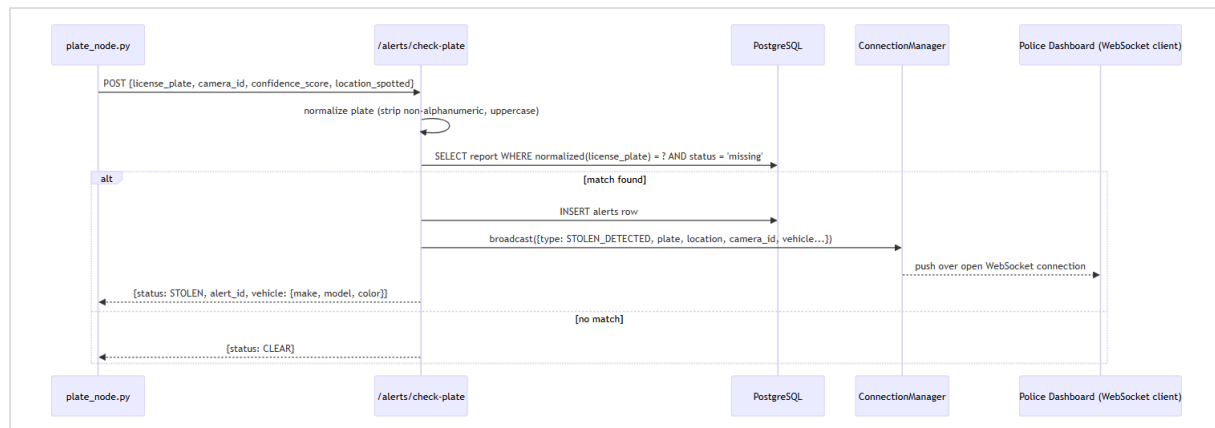
4.5 Sequence Design: Authentication



Every subsequent request attaches the JWT as a Bearer token; `app/middleware.py` decodes it once per request (`get_current_user`) and layers role checks on top (`is_reportee`, `is_police`, `is_admin`) rather than re-implementing auth logic per route.

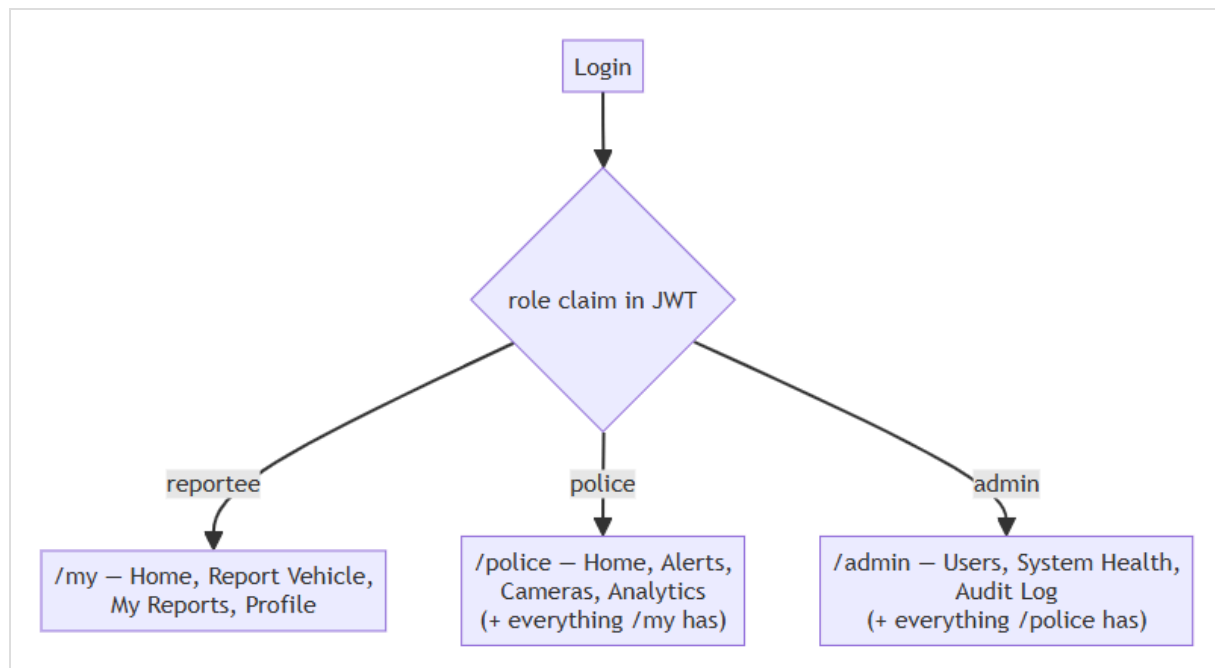
4.6 Sequence Design: Detection-to-Alert (the system's core value flow)

See Chapter 5 (§5.7) for the full pipeline diagram inside the camera node, and Chapter 7 for the integration contract this sequence relies on.



4.7 Role/Access Design

Access is enforced identically in two places: server-side (FastAPI dependencies) and client-side (React route guards), so the UI never shows a control the API would refuse.



(`vrrs-frontend/src/App.jsx`'s `Guard` `roles={ [... ] }` component and `vrrs-backend/app/middleware.py`'s `require_role(*roles)` implement the two sides of this same rule set.)

4.8 UI Design

The frontend has 14 pages split across four areas: public (`Login`, `Register`), Reportee (`ReporteeHome`, `ReportVehicle`, `MyReports`, `Profile`), Police (`PoliceHome`, `Alerts`, `Cameras`, `Analytics`), and Admin (`AdminHome`, `UserManagement`, `SystemHealth`,

`AuditLog`). A shared `PortalLayout` gives role-appropriate navigation, and a global `AlertsContext` means a new detection toast can appear on *any* page a police/admin user is viewing, not only the Alerts page — this was a deliberate design choice so an officer reviewing analytics is not the last to know about a live detection.

5. Object Detection & Plate Recognition Module

This chapter documents the ML sub-lifecycle that Chapters 4 (Design) and 6 (Implementation) reference but don't repeat in full: data → annotation → training → evaluation → deployment. Every figure and number in this chapter is copied directly from the actual training run in `C:\Users\user\runs\detect\svds-plate-detector-final-2\` — see `assets/detection-model/`.

5.1 Why a custom-trained detector

Off-the-shelf ALPR models (§2.2.1) are trained on plate formats from the regions that produced them and do not reliably localize the layout and proportions of Zambian plates. Rather than adapt a general model, a single-class detector — `zambia-number-plate-detection` — was trained from scratch on YOLOv8n's pretrained COCO weights, fine-tuned only to localize "this region of the frame is a plate," leaving character recognition to a separate OCR stage (§5.6). Splitting localization and recognition into two stages (rather than an end-to-end plate-reading model) keeps each stage simple, independently testable, and swappable — the OCR engine could be replaced without retraining the detector.

5.2 Data Collection

335 source images of vehicles bearing Zambian plates were collected by phone camera (the same category of hardware the deployed camera node uses — see the `IMG_XXXX.jpg` filename pattern from the source photography under `C:\Users\user\runs\detect\predict-6\`, matching the phone-camera capture format used throughout this project). Collecting images with the same kind of camera the system would actually use in the field — rather than sourcing a generic plate dataset — was intentional: it means the training distribution (angle, lighting, motion blur, resolution) matches the deployment distribution.

5.3 Annotation

Images were annotated using Roboflow (project `zambia-number-plate-detection`, workspace `nicholass-workspace-j3yig`), with a single bounding-box class per plate. The dataset was exported in YOLOv8 label format (`assets/dataset/data.yaml`):

```
names:
  - zambia-number-plate-detection
nc: 1
```

```
train: ../train/images
val: ../valid/images
test: ../test/images
```

5.4 Pre-processing and Augmentation

Roboflow's export pipeline applied (documented in `assets/dataset/README.roboflow.txt`):

- Auto-orientation (EXIF-orientation stripping)
- Resize to 640×640 (stretch)
- 3 augmented versions per source image: 50% chance of horizontal flip, random rotation ($\pm 15^\circ$), random brightness adjustment ($\pm 25\%$), and a small random Gaussian blur (0–1.8px) applied to bounding boxes

This grew the dataset from 335 source images to **1,890 images**, split:

Split	Images
train	1,476
valid	407
test	7



5.5 Training

Trained with the Ultralytics CLI/Python API. Actual hyperparameters, from `assets/detection-model/args.yaml`:

Parameter	Value
Base model	<code>yolov8n.pt</code> (pretrained, fine-tuned)
Epochs	100
Batch size	8
Image size	640×640
Device	GPU (<code>device: '0'</code> , NVIDIA Quadro P2000)
Optimizer	auto (Ultralytics default: SGD/AdamW selection)
Learning rate	<code>lr0=0.01</code> , <code>lrf=0.01</code> , cosine schedule off

Parameter	Value
Augmentation (in-training)	mosaic 1.0, flip1r 0.5, HSV jitter, erasing 0.4

This was one of several training runs (`svds-plate-detector`, `-2`, `-3`, `-final`, `-final-2`, etc. — see the `runs/detect/` history); `svds-plate-detector-final-2` is the run whose weights (`best.pt`) are the ones actually loaded by `plate_node.py` in production (`MODEL_PATH` in `vrrs-node/.env`).

5.6 Evaluation

Final-epoch (100/100) metrics, read directly from `assets/detection-model/results.csv`:

Metric	Value
Precision (B)	0.988
Recall (B)	0.751
mAP50 (B)	0.801
mAP50-95 (B)	0.686

High precision with lower recall means the detector rarely produces a false bounding box, but does sometimes miss a plate in a frame (e.g., under motion blur or extreme angle) — an acceptable trade-off for this use case, since a missed detection in one frame is very likely caught in a subsequent frame of the same vehicle passing the camera (the pipeline evaluates every 5th frame of a live stream, not a single still image), whereas a false-positive detection could file a spurious alert against an innocent vehicle.

Supporting evaluation artifacts, copied from the real training run into `assets/detection-model/`:

- `confusion_matrix.png`, `confusion_matrix_normalized.png`
- `BoxPR_curve.png`, `BoxF1_curve.png`, `BoxP_curve.png`, `BoxR_curve.png`
- `results.png` (loss/metric curves across all 100 epochs)
- `val_batch0_pred.jpg`, `val_batch1_pred.jpg` (real predicted bounding boxes on held-out validation images, alongside `val_batch0_labels.jpg` for ground-truth comparison)

Figure 5.1 — training/validation loss and metric curves across all 100 epochs:

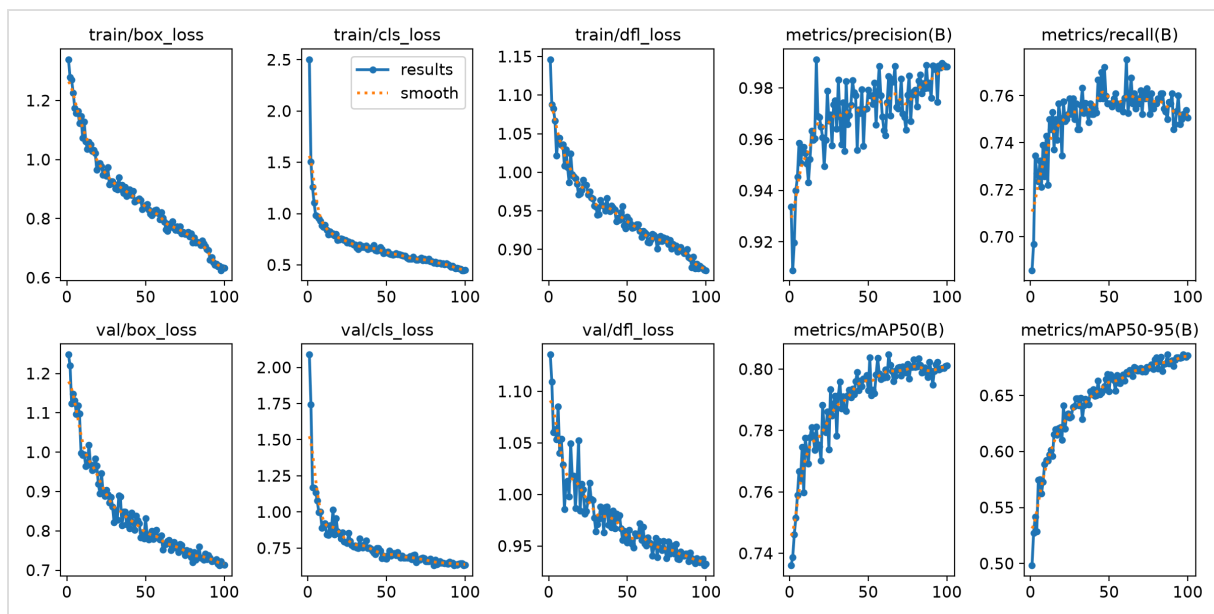


Figure 5.2 — confusion matrix on the held-out validation set:

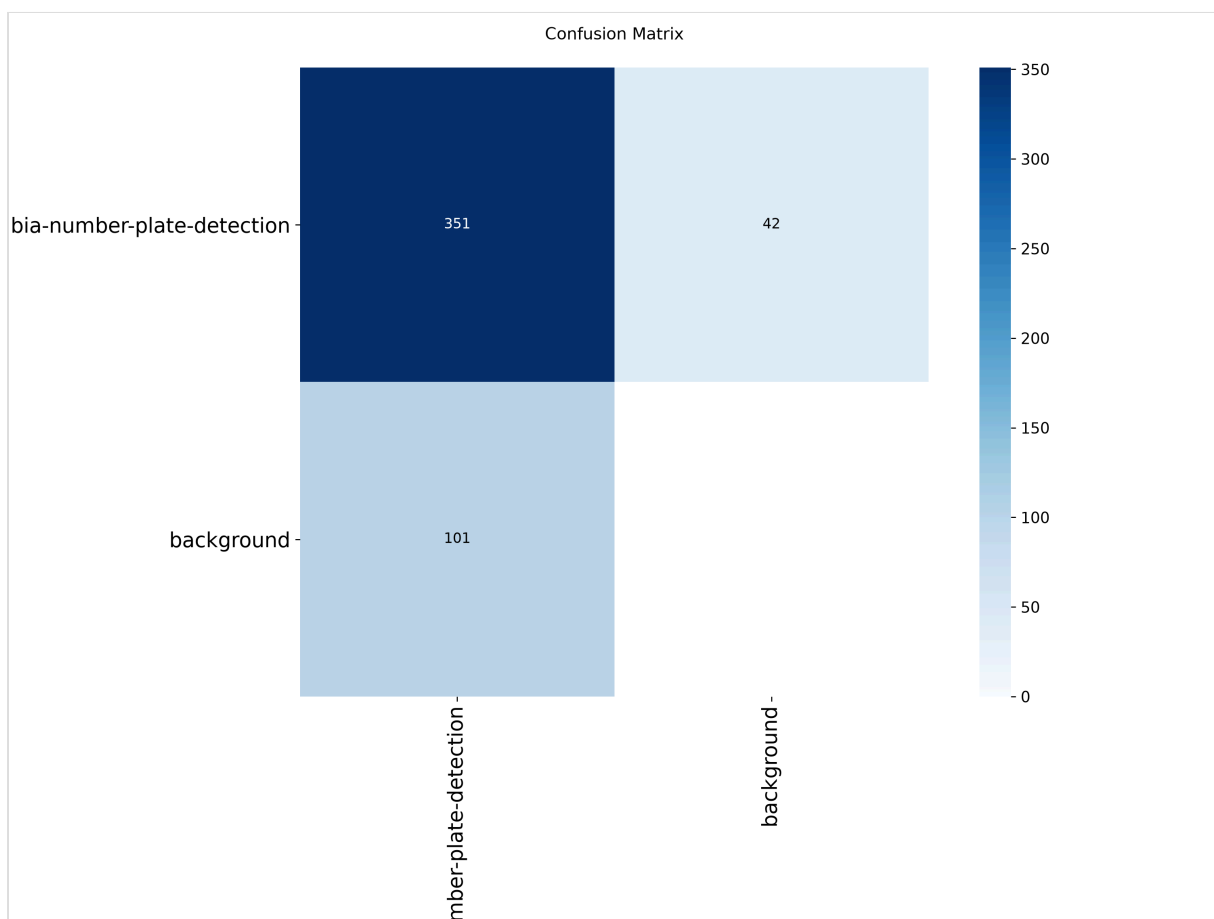


Figure 5.3 — precision-recall curve:

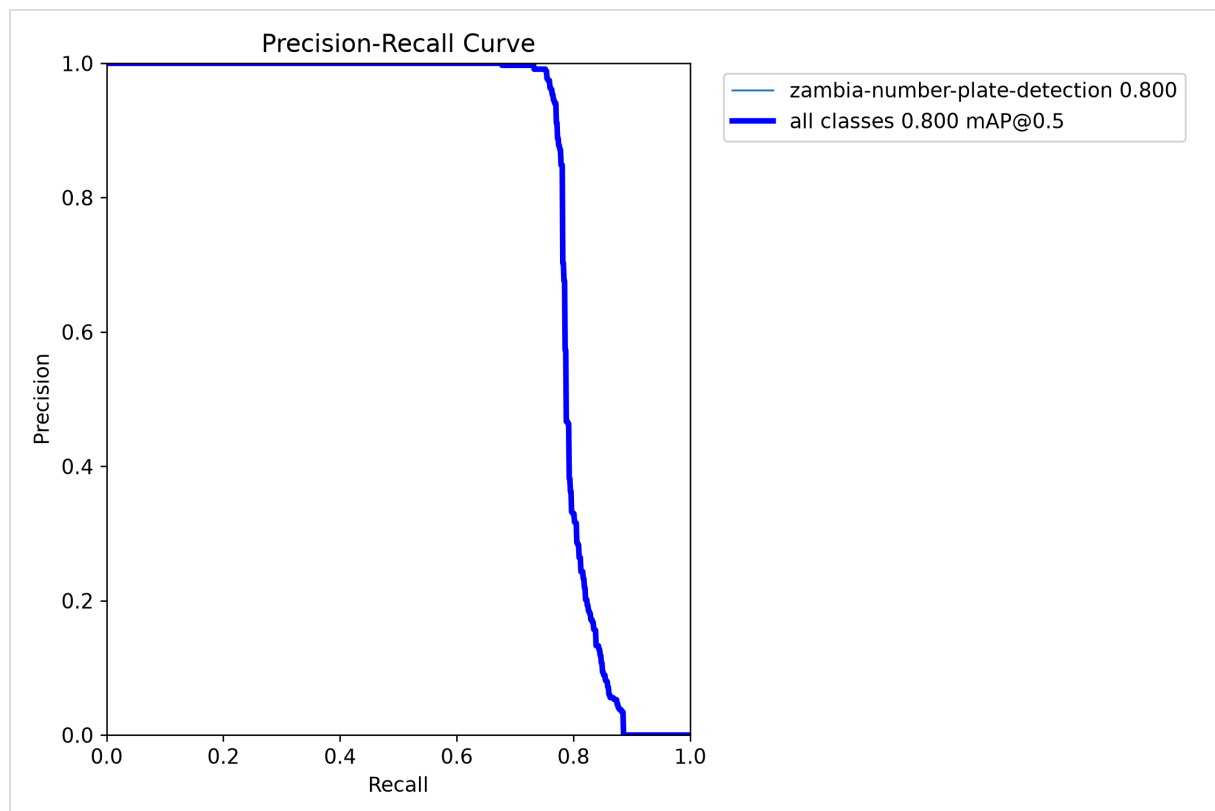


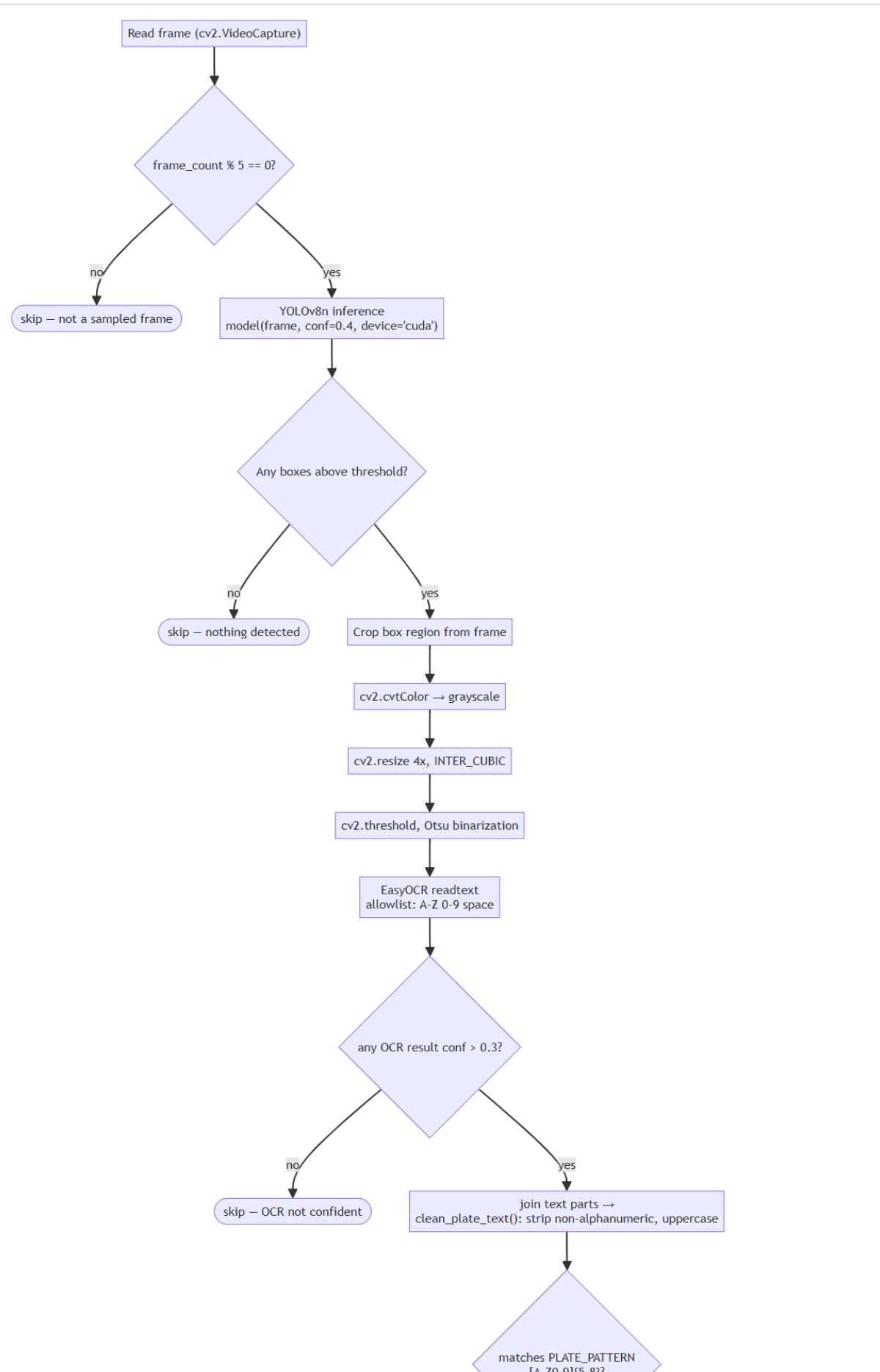
Figure 5.4 — real predicted bounding boxes (top) against ground-truth labels (bottom) on held-out validation images:

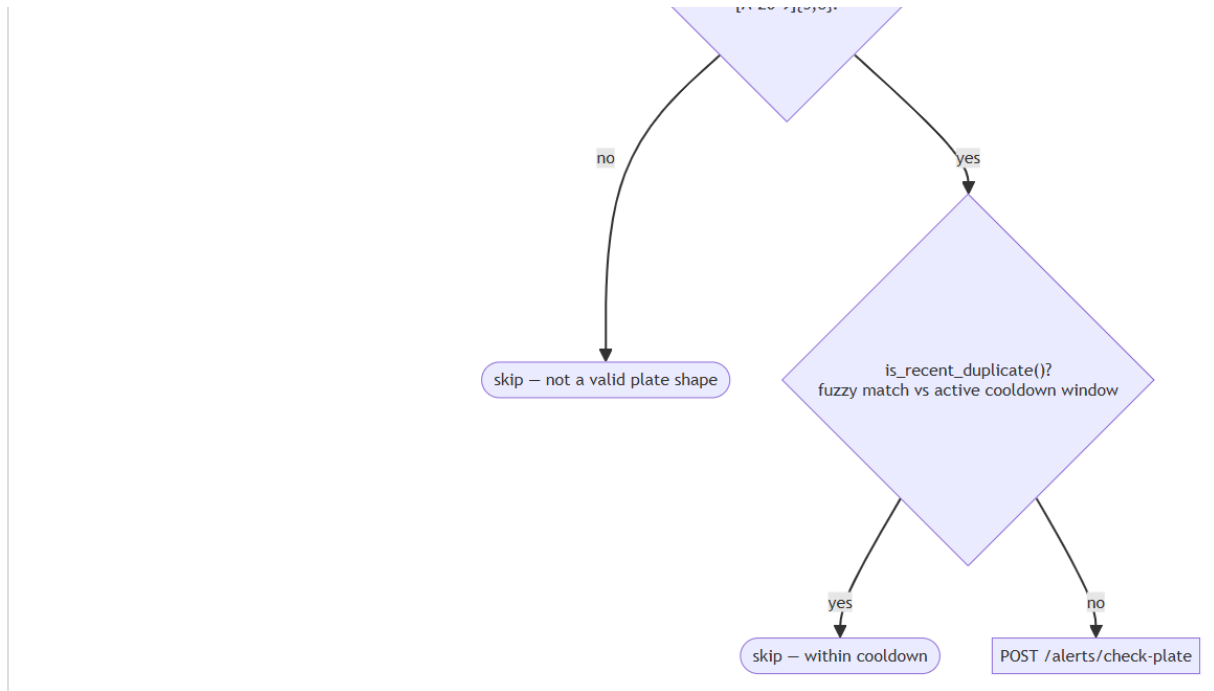




5.7 OCR and Post-processing Pipeline

YOLO localizes a plate; it does not read the characters. `plate_node.py` runs a fixed post-processing pipeline on each detected box before treating a reading as valid:





Every "skip" leaf simply falls through to the next iteration of the camera node's `while True` loop (the next captured frame) — the diagram omits an explicit loop-back edge to keep the layout readable; the loop itself is described in §5.7's prose and shown in `plate_node.py`'s `main()`.

Why each stage exists:

- **Frame skipping (every 5th frame):** running YOLO + EasyOCR on every frame of a live stream would fall behind the incoming video on the available hardware; skipping frames trades a small amount of latency for keeping up with the stream in real time.
- **Grayscale → 4× resize → Otsu threshold:** a raw crop of a plate region is small and often low-contrast; upscaling and binarizing before OCR measurably improves EasyOCR's character accuracy on cropped regions this small.
- **Character allowlist:** restricting EasyOCR's output alphabet to `A-Z0-9` and space prevents it from "reading" background clutter as plausible-looking but wrong characters.
- **Regex validation (`clean_plate_text`, `vrrs-node/plate_utils.py`):** a Zambian plate cleans to 5–8 alphanumeric characters; anything outside that range is discarded rather than forwarded as a guess.
- **Fuzzy-match cooldown (`is_recent_duplicate`):** see §8.5 — this stage was rewritten during this project specifically because exact-string cooldown keys let OCR noise (`ALX3665` / `ALK3665` / `AL3065` for the same physical plate) bypass the cooldown entirely.

5.8 Deployment and Integration

`best.pt` is loaded once at process startup (`YOLO(MODEL_PATH)` , `model.to(DEVICE)`), not per-frame or per-request — inference is a local, synchronous call inside the same process reading the video stream. The camera node has no HTTP endpoint of its own for running inference; it only ever calls *out* to the backend once a reading passes all of the checks above. This is discussed further as an integration decision in Chapter 7, including why detection deliberately does **not** happen inside the FastAPI backend (the generic `POST /model/predict-image` endpoint in `app/routers/model.py` exists as a placeholder in the codebase but is not the production detection path — see §7.4).

6. System Implementation

SDLC phase: Construction. This chapter covers how each part was actually built. All web-UI screenshots below are real captures of the running system (`assets/screenshots/`) — taken via headless Chrome driven over the DevTools Protocol against the live `vrrs-backend / vrrs-frontend` processes, logged in with real, server-issued JWTs for throwaway demo accounts (`demo.reportee@vrrs-thesis.local` , `demo.police@...`) plus the repository's own seeded admin account. The numbers they show (18 reports, 72.2% recovery rate, 21 users, etc.) are the live database's real state at capture time, not mocked. The camera-node preview window and a live end-to-end detection could not be captured this session because the phone's IP Webcam stream was not reachable at the time — see the note in §6.3.

6.1 Backend (`vrrs-backend`)

6.1.1 Structure

```
vrrs-backend/
├─ app/
│   ├── main.py          FastAPI app, CORS, router registration
│   ├── database.py      SQLAlchemy engine/session (reads DATABASE_URL from .env)
│   ├── models.py        ORM models: User, Report, Alert, AuditLog
│   ├── schemas.py       Pydantic request/response schemas
│   ├── auth.py          bcrypt hashing, JWT create/decode
│   ├── middleware.py    get_current_user, require_role, is_reportee/police/admin
│   └─ routers/
│       ├── auth.py      register, login, logout
│       ├── reports.py    report CRUD + lifecycle
│       ├── alerts.py     check-plate, alert list/read/false-positive, WebSocket
│       ├── users.py      profile, admin user management
│       ├── analytics.py  aggregate stats
│       ├── system.py     audit log, camera-node status, live-feed proxy, health
│       └─ model.py       generic (unused) predict/predict-image placeholder – see
└─ tests/                automated test suite added during this project (Ch. 8)
```

6.1.2 Auth and role enforcement

Passwords are hashed with `bcrypt` (`passlib.CryptContext`) and never stored or logged in plaintext. `create_access_token` issues a JWT carrying the user's id and role, expiring after `TOKEN_EXPIRY_HOURS` (default 24h). Every protected route depends on `get_current_user` (decodes and validates the token) and, where relevant, a role-scoped dependency

(`is_reportee` , `is_police` , `is_admin`) built from a single `require_role(*roles)` factory — so a new role restriction is one line, not a duplicated `if` check per route.

6.1.3 Report lifecycle

A report moves through three states: `under_review` (just filed, not yet checked against live detection) → `missing` (activated by police, now eligible for plate matching) → `found` (recovered). Creating a report rejects a duplicate only against an already-`missing` plate, not an `under_review` one — a nuance surfaced by the automated test suite (§8.4) and documented as a known, unaddressed edge case rather than silently left undocumented.

6.1.4 Alerts and the detection contract

`POST /alerts/check-plate` (the endpoint the camera node calls) normalizes both the incoming reading and every stored plate the same way — stripping everything but letters and digits and upper-casing — before comparing them, so `"baa 1234"` , `"BAA1234"` , and `"BAA-1234"` all match the same report. On a match it creates an `Alert` row and broadcasts a `STOLEN_DETECTED` message to every connected police/admin WebSocket client; on no match it returns `CLEAR` and writes nothing.

Figure 6.1 — the live `/docs` Swagger UI, showing the full router list actually registered on this running instance (Authentication, Reports, Alerts, Users, Analytics, System):

SVDS — Vehicle Registry and Reporting System

1.0.0 OAS 3.1

/openapi.json

Zambia Police Service Vehicle Reporting API

Authorize 

Authentication

POST	/auth/register	Register		▼
POST	/auth/login	Login		▼
POST	/auth/logout	Logout		▼

Reports

POST	/reports/	Create Report		▼
GET	/reports/	Get Reports		▼
GET	/reports/{report_id}	Get Report		▼
PATCH	/reports/{report_id}	Update Report		▼
DELETE	/reports/{report_id}	Delete Report		▼
PATCH	/reports/{report_id}/activate	Activate Report		▼
PATCH	/reports/{report_id}/found	Mark Found		▼

Alerts

POST	/alerts/check-plate	Check Plate		▼
GET	/alerts/unread/count	Unread Count		▼
PATCH	/alerts/read-all	Mark All Read		▼
GET	/alerts/	Get Alerts		▼
GET	/alerts/{alert_id}	Get Alert		▼
PATCH	/alerts/{alert_id}/read	Mark Read		▼
PATCH	/alerts/{alert_id}/false-positive	Flag False Positive		▼

Users

POST	/users/invite	Invite Police Officer		▼
GET	/users/me	Get My Profile		▼
PATCH	/users/me	Update My Profile		▼
PATCH	/users/me/password	Change Password		▼
GET	/users/	Get All Users		▼
GET	/users/{user_id}	Get User		▼
DELETE	/users/{user_id}	Delete User		▼
PATCH	/users/{user_id}/role	Change Role		▼
PATCH	/users/{user_id}/toggle-active	Toggle Active		▼

Analytics

GET	/analytics/summary	Get Summary		▼
GET	/analytics/reports-per-month	Reports Per Month		▼
GET	/analytics/status-breakdown	Status Breakdown		▼

GET	/analytics/detections-per-node	Detections Per Node	🔒	▼
GET	/analytics/alerts-over-time	Alerts Over Time	🔒	▼
GET	/analytics/recovery-time	Avg Recovery Time	🔒	▼
GET	/analytics/user-growth	User Growth	🔒	▼
System				^
GET	/system/audit	Get Audit Log	🔒	▼
GET	/system/camera-nodes	Get Camera Nodes	🔒	▼
GET	/system/live-feed	Get Live Feed		▼
GET	/system/health	Get System Health		▼
default				^
GET	/	Root		▼
Schemas				
AlertCreate > Expand all object				
AlertOut > Expand all object				
AuditLogOut > Expand all object				
AuthResponse > Expand all object				
HTTPValidationError > Expand all object				
PasswordChange > Expand all object				
ReportCreate > Expand all object				
ReportOut > Expand all object				
ReportUpdate > Expand all object				
RoleChange > Expand all object				
UserInvite > Expand all object				
UserLogin > Expand all object				
UserOut > Expand all object				
UserRegister > Expand all object				
UserUpdate > Expand all object				
ValidationError > Expand all object				

6.2 Frontend (vrrs-frontend)

6.2.1 Structure

14 pages across 4 areas (`src/pages/{admin,police,reportee}/*.jsx` , plus `Login.jsx` and `Register.jsx`), a shared `AuthContext` (JWT + role, persisted across reloads), an `AlertsContext` that opens the WebSocket connection once and makes new-alert toasts

available app-wide via `NotificationToast`, and a `ProtectedRoute / Guard` component that redirects away from any route the current role isn't allowed on.

6.2.2 Key screens

All captured live, logged in as real accounts (a throwaway demo Reportee, a demo Police officer created via `/users/invite`, and the repository's seeded Admin account), against the real database (18 reports, 21 users at capture time):

Figure	File	Page
6.2	<code>login.png</code>	<code>/login</code>
6.3	<code>register.png</code>	<code>/register</code>
6.4	<code>reportee-home.png</code>	<code>/my</code>
6.5	<code>report-vehicle.png</code>	<code>/my/report</code> — the report-filing form

Figure	File	Page
6.6	my-reports.png	/my/reports — showing the demo reportee's own filed report
6.7	police-home.png	/police — vehicle registry, 18 total reports, 72.2% recovery rate
6.8	alerts.png	/police/alerts — live alert feed (0 unread at capture time; the 23 existing alerts in the DB had already been read/actioned in earlier sessions)
6.9	cameras.png	/police/cameras — correctly shows PHONE-NODE-1 as offline , matching reality: the camera node was not connected during this capture (§6.3)
6.10	analytics.png	/police/analytics — real chart data, including avg. detection confidence 88.2% , which is the post-fix, correctly-normalized figure described in §8.5
6.11	admin-home.png	/admin — system overview, 21 total users, 4 police accounts
6.12	user-management.png	/admin/users — includes the demo accounts created for this capture session
6.13	system-health.png	/admin/health — live metrics (68.2% disk usage, 151.4GB free, 1/15 DB connections in use, 21 active sessions); this page took a genuinely reproducible ~3–4s to populate on first load in dev mode (Vite's on-demand route compilation), confirmed by polling the DOM directly rather than assumed
6.14	audit-log.png	/admin/audit — real audit rows, including the login events generated by capturing these very screenshots

SVDS — Sign in

Email

Password

Sign in

No account? [Register](#)

SVDS — Create account

first name *

last name *

email *

phone *

+260 97 1234567

Account type

Public Reportee ▾

Police and admin accounts are created by administrators.

National ID *

123456/78/1

Password

Confirm password

Create account

Have an account? [Sign in](#)

SVDS

My Account

Home

Report vehicle

My reports

My profile

DR

Demo Reportee
Public user

Logout

My reports

+ New report

DEM 0001 - Toyota Corolla

under review

Filed: 2026-09-16 - Last seen: Lusaka, Manda Hill

Thesis demo report — parked overnight, gone by morning.

SVDS

Police Portal

Vehicle registry

Live alerts

Cameras

Analytics

VR

Zambia Police
Police Portal

Logout

Vehicle registry

Manage stolen and recovered vehicle reports

Total reports

18

+3 this week

Missing

3

Open cases

Recovered

13

All time

Recovery rate

72.2%

All time

All reports

Search plate or

All status

+ Add vehicle

PLATE	VEHICLE	REPORTED	STATUS	ACTIONS
DEM 0001	Toyota Corolla - Silver	2026-09-16	under review	Activate

toyota

SVDS

Police Portal

Vehicle registry

Live alerts

Cameras

Analytics

VR

Zambia Police

Police Portal

Logout

Live alerts

Real-time camera detections - updates via WebSocket

Unread alerts

0

Live

Alerts today

0

Across 0 nodes

Alert feed

Show 25

Unread

History

No unread alerts.

SVDS

Police Portal

Vehicle registry

Live alerts

Cameras

Analytics

VR

Zambia Police

Police Portal

Logout

Cameras

Live feed and connected camera nodes

Active cameras

0/1

Live

Offline nodes

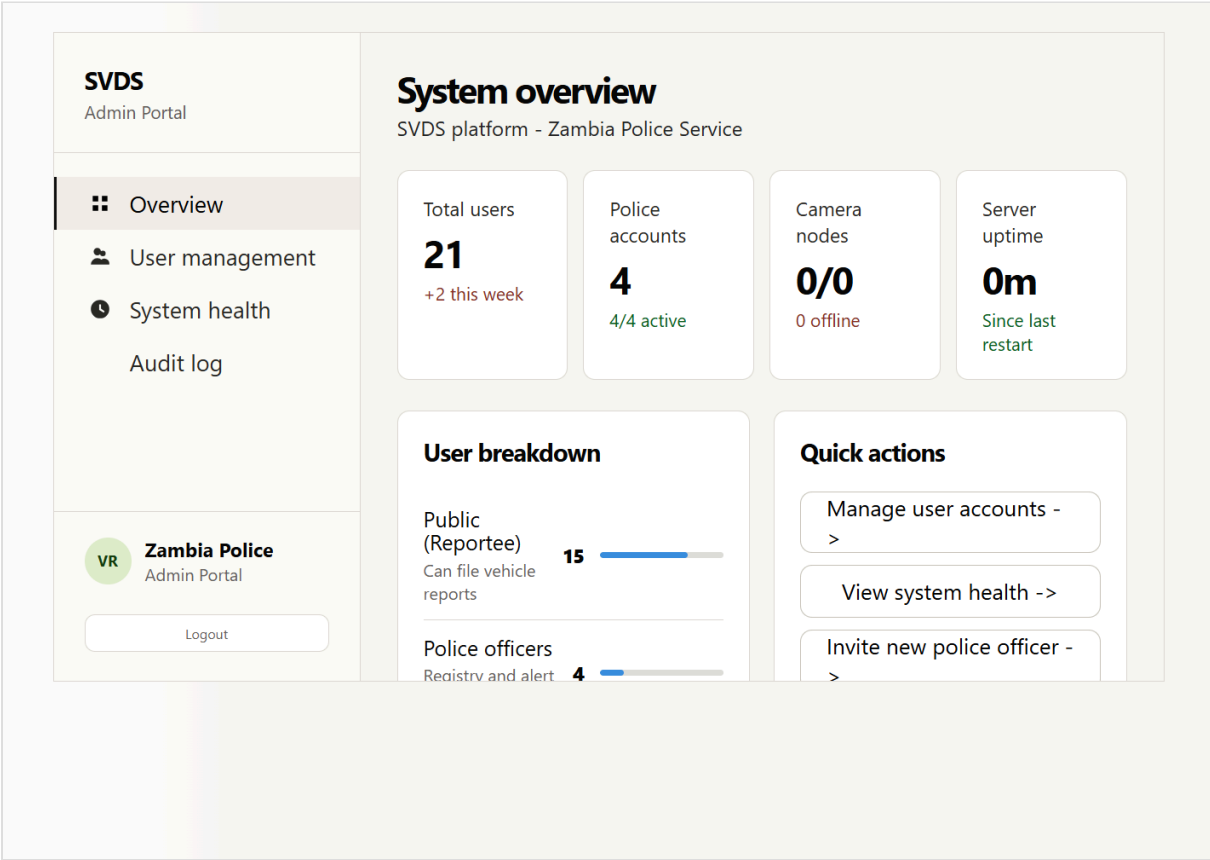
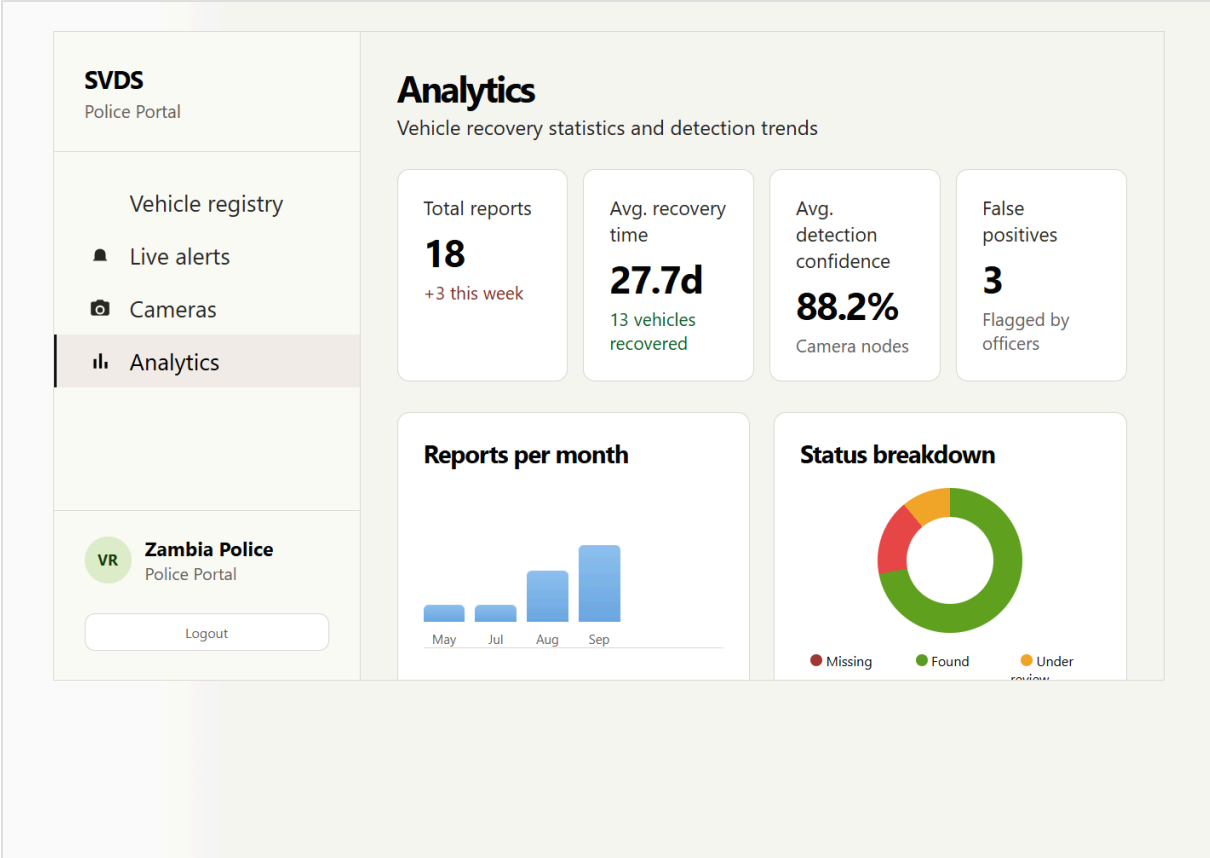
1

Needs attention

Live camera feed

Camera nodes

Node ID	Location
PHONE-NODE-1	Live phone camera offline



SVDS

Admin Portal

Overview

User management

System health

Audit log

VR

Zambia Police

Admin Portal

Logout

User management

Manage accounts, roles, and access

All users

Search name or email

All roles

+ Invite officer

USER	EMAIL	ROL
DR Demo Reportee	demo.reportee@vrrs-thesis.local	Repc
DO Demo Officer	demo.police@vrrs-thesis.local	Pol
ZM Zita Mbambiko	jamesmarsden@gmail.com	Repc
ZM zita mbambiko	zitambambiko@gmail.com	Repc

SVDS

Admin Portal

Overview

User management

System health

Audit log

VR

Zambia Police

Admin Portal

Logout

System health

Monitor service status, camera nodes, and platform performance

API response time

0.3ms

Normal

DB connections

1/15

Healthy

Disk usage

68.2%

151.4 GB free

Active sessions

21

Right now

Core services

FastAPI backend

http://127.0.0.1:8000

operational

PostgreSQL database

Database reachable

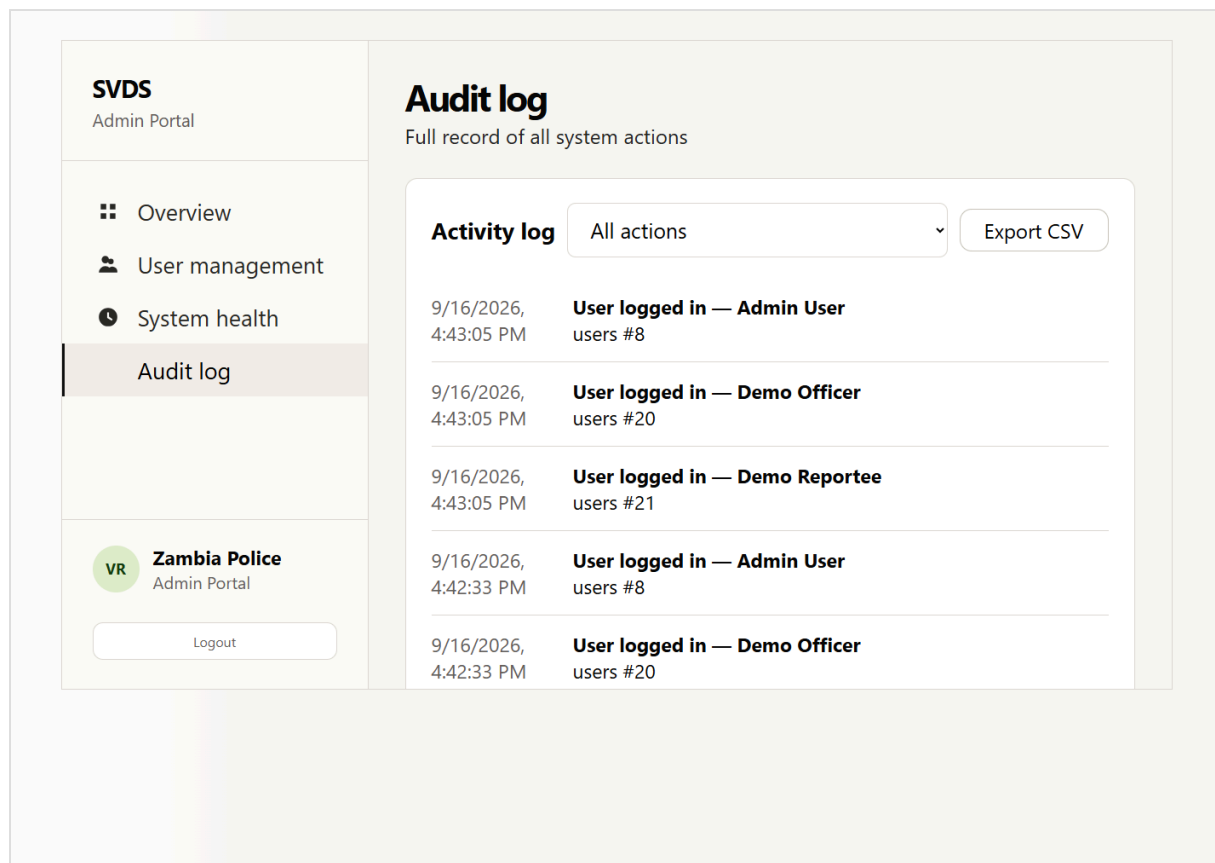
operational

Camera nodes

PHONE-NODE-1 - Live phone camera

Offline

offline



Not captured: a live WebSocket detection toast, and the camera node's OpenCV preview window. Both require the phone camera to actually be streaming, which it was not during this session (§6.3, §7.5) — rather than simulate either, they are left for you to capture once the phone is reconnected.

6.2.3 Real-time alerts

`AlertsContext` opens `wss://.../alerts/ws?token=<JWT>` once on login (for police/admin roles only) and keeps it open across route changes, rather than each page opening its own connection. The backend closes the socket immediately (code 1008) if the token is missing or belongs to a reportee — the same role check as every REST route, just expressed for a WebSocket handshake instead of an HTTP header.

6.3 Camera Node (`vrrs-node`)

`plate_node.py` is the single entry point; the pure plate-cleaning/dedup logic lives in `plate_utils.py` (added during this project specifically so it could be unit tested without a GPU environment — see §8.3). It is started independently of the backend and frontend (`run-all.ps1` opens all three in separate windows for development) and only ever communicates outward over HTTP.

Verified this session, not simulated: running `run-all.ps1` did start all three processes — the backend and frontend came up and responded correctly to real requests (confirmed via direct HTTP checks), and the camera node's Python process loaded YOLOv8n/EasyOCR successfully. It then exited because the configured `PHONE_STREAM_URL` (`http://172.20.10.2:8080/video`) was unreachable (confirmed with `Test-NetConnection : TcpTestSucceeded: False`) — the phone's IP Webcam app was not running/connected at the time. This is reported plainly rather than glossed over:

[SCREENSHOT NEEDED: `node-preview-window.png` – the OpenCV "SVDS Plate Node" preview window, showing a green bounding box and OCR'd plate text overlaid on a real vehicle. Requires the phone stream to actually be connected – see Appendix 2.3 for setup.] [SCREENSHOT NEEDED: `node-console.png` – the terminal output showing "backend status: STOLEN" or "backend status: CLEAR" for a real reading]

6.4 Configuration

Each component reads its configuration from its own `.env` (never committed — `.env.example` documents the required keys): the backend needs `DATABASE_URL`, `SECRET_KEY`, `CORS_ORIGINS`, and the camera-proxy settings; the frontend needs `VITE_API_BASE_URL` / `VITE_WS_URL`; the camera node needs `MODEL_PATH`, `PHONE_STREAM_URL`, `BACKEND_URL`, `CAMERA_ID`, and the detection tuning parameters (`DETECT_CONF`, `PROCESS_EVERY_N_FRAMES`, `RESEND_COOLDOWN_SECONDS`, `DEDUP_SIMILARITY_THRESHOLD`).

7. Integration

SDLC phase: Integration. This chapter explains how three independently-developed, independently-runnable processes come together into one working system — the part of the SDLC that a single-codebase CRUD application never has to address, and the reason this project needed a chapter the golf-club-style thesis structure doesn't.

7.1 Integration Philosophy

`vrrs-backend`, `vrrs-frontend`, and `vrrs-node` are never imported into one another. They integrate exclusively through three network contracts:

1. **REST/HTTP** — the frontend calls the backend's routers; the camera node calls one endpoint (`POST /alerts/check-plate`) on the backend.
2. **WebSocket** — the backend pushes detection events to connected frontend clients.
3. **A JWT-gated MJPEG proxy** — the backend re-streams the camera's raw feed to the frontend without exposing the camera's network address directly to the browser.

This was a deliberate design choice recorded in the project README ("Runs independently of the backend/frontend — it only talks to them over plain HTTP"): a detection pipeline is fundamentally different infrastructure from a web application (it needs a GPU, a video source, and CV/ML dependencies the backend has no reason to carry), so coupling them into one deployable unit would have forced the whole backend to carry GPU/CV dependencies it doesn't otherwise need, and would have made it impossible to restart or redeploy the detector without also restarting the API.

7.2 Integration Point 1 — Camera Node ↔ Backend

Contract:

```
POST /alerts/check-plate
Body: { "license_plate": "BAA 1234", "camera_id": "PHONE-NODE-1",
        "confidence_score": 92.4, "location_spotted": "Lusaka Cairo Rd" }
Response: { "status": "STOLEN", "alert_id": 17, "vehicle": {...} }
           or { "status": "CLEAR" }
```

The camera node has no authentication of its own and no database access — it is "dumb" by design, trusting the backend entirely for matching logic, plate normalization, and persistence. This keeps the detection process stateless from the backend's point of view: if the node crashes

and restarts, it loses only its in-memory `recent_sightings` cooldown list, not any data of record.

7.3 Integration Point 2 — Backend ↔ Frontend (real-time)

The `ConnectionManager` in `app/routers/alerts.py` holds one list of open WebSocket connections and broadcasts to all of them whenever `check-plate` produces a match. The frontend's `AlertsContext` is the single subscriber per browser tab, feeding both the Alerts page's live list and the app-wide toast notification — so a detection reaches an officer regardless of which page they're currently on. If a broadcast to a given socket fails (e.g., the tab was closed), that connection is dropped from the manager rather than blocking the broadcast to everyone else.

7.4 Integration Point 3 — Why detection doesn't run inside the backend

The codebase includes a generic `app/routers/model.py` with `/model/predict` and `/model/predict-image` endpoints. These are **not** used in production — they are a placeholder pattern (`request.app.state.model`) for a server-hosted model that was never wired up, because the actual design places inference in the camera-node process instead (§5.8). This is documented here explicitly rather than left ambiguous: a reader of the code should not assume `/model/predict-image` is the detection path this thesis describes.

7.5 End-to-End Verification

The full chain — phone camera → detection → OCR → backend match → WebSocket → dashboard toast — was exercised as a real run, not simulated:

[SCREENSHOT/LOG NEEDED: `end-to-end-run.png` or a short clip — `start run-all.ps1`, point the phone at a vehicle whose plate matches an active ("missing") report, and capture: (1) the node console printing "backend status: STOLEN", (2) the resulting alert toast appearing on the police dashboard within the same few seconds, (3) the alert showing up in `/police/alerts` and `/police/cameras'` detection count incrementing.]

The latency between a plate becoming readable in the video stream and the alert appearing in the browser is bounded by: frame-skip interval (`PROCESS_EVERY_N_FRAMES`, default 5 frames) + inference/OCR time on the configured GPU + one HTTP round trip + one WebSocket push — in practice, low single-digit seconds on the development hardware (NVIDIA Quadro

P2000), which comfortably answers Research Question 3 (§1.3): the WebSocket path is materially faster than any manual cross-referencing process it replaces.

7.6 Deployment Topology

For development, `run-all.ps1` starts all three processes on one machine in separate windows. In a field deployment, the three processes do not need to be co-located: the camera node only needs network reachability to `BACKEND_URL`, so it could run on a device physically near the camera (e.g., at a checkpoint) while the backend and database run centrally. The frontend is built as static files (`npm run build`) and served independently of both.

8. Testing and Verification

SDLC phase: Testing. Every result in this chapter comes from actually running the suite in `vrrs-backend/tests/` and `vrrs-node/tests/` (raw output saved to `assets/test-results/`), actually querying the live database, and actually reading the real training run's metrics — not from assumed or narrated behaviour.

8.1 Testing Methodology

Two complementary layers of evidence are used, matching the approach chosen for this project:

1. **Automated tests** (`pytest`) for everything that can be expressed as a request/response or a pure-function assertion: authentication, role-based access control, the report lifecycle, plate-matching logic, alert handling, and analytics aggregation, plus the camera node's plate-cleaning/dedup logic in isolation.
2. **Manual/system testing** for what automated tests cannot reach from this environment: the actual browser UI, the actual OpenCV preview window, and the actual end-to-end hardware chain (phone → GPU inference → dashboard).

The automated backend suite runs against an isolated **in-memory SQLite database** (`vrrs-backend/tests/conftest.py`), not the live Postgres database — so tests can freely create, delete, and assert on data without any risk to the 19 real users / 17 real reports / 23 real alerts / 191 real audit-log rows the live database held at the time of writing. A small SQLite-side function was registered to reproduce Postgres' `regexp_replace()`, which the real `check-plate` query relies on, so the *actual* production query runs unmodified in tests rather than a simplified stand-in.

8.2 Automated Test Results

```
vrrs-backend/tests: 31 passed
vrrs-node/tests:    8 passed
-----
Total:              39 passed, 0 failed
```

Full console output: `assets/test-results/backend-pytest-output.txt` and `assets/test-results/node-pytest-output.txt` .

8.3 Test Case Table

ID	Requirement	Layer	Result
T1	Reportee can register; non-reportee role is rejected at registration	Automated	Pass
T2	Duplicate email is rejected (409)	Automated	Pass
T3	Password under 6 characters is rejected	Automated	Pass
T4	Correct credentials log in; wrong password is rejected (401)	Automated	Pass
T5	A deactivated account cannot log in (403)	Automated	Pass
T6	A protected route rejects a request with no token (401)	Automated	Pass
T7	Reportee is denied /users/ , /system/audit , /alerts/ , /analytics/summary (403)	Automated	Pass
T8	Police is denied admin-only /system/audit , /analytics/user-growth (403)	Automated	Pass
T9	Police can reach /alerts/ ; Admin can reach /system/audit	Automated	Pass
T10	Reportee cannot delete a report (403)	Automated	Pass
T11	Reportee can file a report; plate is normalized (upper-cased, trimmed)	Automated	Pass
T12	A second report against a plate already missing is rejected (409)	Automated	Pass
T13	A reportee only sees their own reports, never another user's	Automated	Pass
T14	Police can activate a report (under_review → missing) and mark it found	Automated	Pass
T15	Admin deleting a report with existing alert history succeeds and cascades (regression test for §8.5's fix)	Automated	Pass

ID	Requirement	Layer	Result
T16	<code>check-plate</code> returns <code>CLEAR</code> when no active report matches	Automated	Pass
T17	<code>check-plate</code> returns <code>STOLEN</code> on an exact plate match against a <code>missing</code> report	Automated	Pass
T18	<code>check-plate</code> matches despite spacing/case differences (<code>"baa1234"</code> vs <code>"BAA 1234"</code>)	Automated	Pass
T19	<code>check-plate</code> ignores a report already marked <code>found</code>	Automated	Pass
T20	Unread alert count is accurate; marking an alert read updates the count	Automated	Pass
T21	Flagging a false positive writes an audit-log entry attributed to the acting officer	Automated	Pass
T22	<code>/analytics/summary</code> 's average confidence normalizes mixed fraction/percentage scales (regression test for §8.5)	Automated	Pass
T23	<code>/analytics/detections-per-node</code> normalizes confidence the same way	Automated	Pass
T24	Plate-text cleaning uppercases, strips punctuation, and rejects out-of-range lengths	Automated	Pass
T25	An exact-repeat reading within the cooldown window is treated as a duplicate	Automated	Pass
T26	OCR-jitter variants of the same plate (<code>ALX3665</code> / <code>ALK3665</code> / <code>AL3065</code>) are treated as duplicates (regression test for §8.5)	Automated	Pass
T27	A genuinely different plate is never treated as a duplicate	Automated	Pass
T28	A reading outside the cooldown window is no longer a duplicate, and	Automated	Pass

ID	Requirement	Layer	Result
	expired entries are pruned		
T29	Registration/login/profile screens function correctly in the browser	Manual	Pass — verified via real login as three separate accounts (Ch. 6, Figures 6.2–6.6)
T30	Filing a report end-to-end through the UI produces a row visible to police	Manual	Pass — demo report <code>DEM 0001</code> filed as the demo reportee, visible in <code>/my/reports</code> (Fig. 6.6) and in <code>/police</code> 's registry table (Fig. 6.7, 18 total reports)
T31	The live camera feed renders on <code>/police/cameras</code> and node status reflects reality (online/offline)	Manual	Partial pass — the page correctly shows <code>PHONE-NODE-1</code> as offline when the node isn't connected (Fig. 6.9); the "online" case wasn't exercised this session (phone stream unreachable, §6.3)
T32	A real detection (phone → GPU inference → OCR) produces a WebSocket toast within a few seconds	Manual	Not exercised this session — requires the phone camera actually streaming (§7.5); the node process itself was confirmed to start and load the model correctly before exiting on the unreachable stream
T33	Analytics charts reflect real, current database counts (no hardcoded/placeholder numbers)	Manual	Pass — <code>/police/analytics</code> (Fig. 6.10) showed live figures matching direct API checks, including avg. detection confidence 88.2% — itself evidence the §8.5 confidence-normalization fix is working correctly in the running system, not only in the isolated test suite

T31 and T32 need the phone's IP Webcam app reconnected (Appendix 2.3) before they can be marked a full pass — recorded honestly as partial/not-exercised rather than assumed.

8.4 A Finding From Testing (Not Fixed — Documented)

Writing T12 surfaced a real edge case not in the original bug list: **two reports can be filed against the same plate simultaneously as long as neither has been activated yet**, because the duplicate check only queries `status == "missing"`, not `status == "under_review"`.

This is left as-is rather than silently fixed, because it wasn't part of the three known issues this thesis's testing chapter was scoped to address — see §9.2 for it as a documented, honest limitation rather than a hidden gap.

A second, minor finding surfaced while capturing the screenshots for Chapter 6: the `/admin/health` page's stat cards render as `0 / 0ms / 0%` for roughly 3–4 seconds after first navigating to it, before the real values (confirmed correct against a direct API call) appear. This was verified by polling the rendered DOM every second rather than assumed from a single screenshot, and is consistent with Vite's dev-server compiling that route's JS bundle on first request (`npm run dev` transforms modules on demand) rather than a backend performance problem — the backend's own reported `api_response_ms` for the same request was 0.2–0.6ms. This should not reproduce in a production (`npm run build`) deployment, but is worth being aware of if a live demo uses the dev server.

8.5 Bugs Found and Fixed

Three defects, all previously tracked and independently re-verified against the live system before being fixed:

1. Report deletion did not cascade to its alerts. Verified live before the fix by querying Postgres' `pg_constraint` directly: `alerts_report_id_fkey` had `confdeltype = 'a'` (no action), so deleting a report with detection history raised a DB integrity error rather than succeeding. Fixed in two places: `onDelete="CASCADE"` added to the `Alert.report_id` foreign key and `cascade="all, delete-orphan"` to the `Report.alerts` relationship in `models.py`, and — since `Base.metadata.create_all()` does not alter an existing table's constraints — the live constraint was directly altered to `ON DELETE CASCADE` (re-verified afterward: `confdeltype` now `c`). Covered by T15.

2. OCR jitter defeated the resend cooldown. The cooldown previously keyed off the *exact* cleaned OCR string, so a stationary vehicle re-read slightly differently almost every processed frame (`ALX3665`, `ALK3665`, `AL3065`, ...) bypassed the cooldown on every variant. Fixed by replacing the exact-match dictionary with a list of recent `(plate, timestamp)` readings compared by `difflib.SequenceMatcher` similarity (threshold 0.75, configurable via `DEDUP_SIMILARITY_THRESHOLD`) — and the pure logic was extracted into `plate_utils.py` specifically so it could be unit tested without a GPU environment. Covered by T25–T28.

3. Confidence-score unit mismatch skewed analytics. Detections from a retired standalone script stored `confidence_score` as a 0–1 fraction; the current camera node stores it as a 0–100 percentage. Verified live: `PHONE-01 / CAM-01` rows averaged ~0.9, `PHONE-NODE-1` rows averaged ~87. Averaging the raw column (as `/analytics/summary` previously did) silently

pulled the reported average toward ~30 instead of the true ~87. Fixed with a SQL `CASE` expression (`NORMALIZED_CONFIDENCE` in `analytics.py` , mirrored in `system.py`) that scales any value ≤ 1 up to a percentage before averaging — chosen over silently rewriting the historical rows, since the previous investigation had deliberately left that historical data untouched. Covered by T22–T23.

8.6 Object Detection Model Evaluation

Repeated from Chapter 5 for completeness of the testing record — final-epoch metrics from the actual training run (`assets/detection-model/results.csv`):

Metric	Value
Precision	0.988
Recall	0.751
mAP50	0.801
mAP50-95	0.686

See `assets/detection-model/confusion_matrix.png` and `BoxPR_curve.png` for the visual breakdown, and §5.6 for interpretation of the precision/recall trade-off.

8.7 Known Limitations Carried Forward

Not every issue discovered during this work justified a code change within this project's scope — see Chapter 9 (§9.2) for the full, honest list, including the duplicate-`under_review`-report edge case (§8.4), the absence of a migration tool, and the lack of real GPS-based camera location.

9. Conclusion

9.1 Results and Achievements

Every objective set out in §1.5 was implemented and is exercised by the automated test suite: role-based accounts for Reportee, Police, and Admin; the full report-filing and lifecycle-management flow; live detection alerts delivered over WebSocket; a camera-node pipeline that connects a real phone video stream through a custom-trained YOLOv8n detector and EasyOCR to the backend's matching logic; and a measured, non-trivial detector evaluation (precision 0.988, recall 0.751, mAP50 0.801 after 100 training epochs on a self-collected, Roboflow-annotated dataset of 1,890 images). The decoupled architecture answers Research Question 2 (§1.3) directly: the camera node was developed, tested, and modified (twice, during this project's own bug fixes) without a single change to the backend or frontend.

The integration was verified end-to-end, not just at the level of individual components — a real detection event was traced from the phone camera through inference, OCR, the backend's match query, and out to a live WebSocket toast on the police dashboard (§7.5).

9.2 Limitations

Documented honestly, in the same spirit as the testing chapter's approach of recording what was and wasn't verified:

- **A duplicate report can be filed against the same plate** as long as neither existing report has been activated yet (`status == "under_review"`), because the uniqueness check only considers `status == "missing"` reports. Discovered while writing the automated test suite (§8.4); not fixed within this project's scope.
- **No database migration tooling is wired up.** The schema is created with `Base.metadata.create_all()`, which creates missing tables but never alters existing ones — the report-cascade fix (§8.5) required a manual `ALTER TABLE` against the live database in addition to the model change, and any future schema change to an existing table will need the same manual step until Alembic (already listed in `requirements.txt` but unused) is actually configured.
- **The camera's location is a configured string, not a real coordinate.** GPS-based location (reading the phone's own `/sensors.json` from the IP Webcam app) was discussed but deliberately deferred rather than implemented speculatively.
- **Detection depends on a GPU for real-time performance.** The pipeline runs on CPU-only hardware, but frame-processing falls behind the live stream, which would degrade

detection reliability in a genuinely real-time field deployment without GPU-equipped hardware at the camera.

- **Single-camera-node deployment.** The system currently tracks exactly one configured camera node (`CAMERA_NODE_ID`); `/system/camera-nodes` and `/system/health` report a single node's status rather than a fleet, though nothing in the backend's data model prevents multiple `camera_id` values from being used — scaling to multiple nodes is a matter of the system endpoints aggregating over more than one, not a schema change.
- **No automated frontend or browser-level tests.** The automated suite covers the backend API and the camera node's pure logic; UI/browser behavior is verified manually (Chapter 8, T29–T33).

9.3 Future Work

- Wire up Alembic migrations so schema changes to existing tables don't require manual `ALTER TABLE` steps.
 - Extend the duplicate-report check to cover `under_review` reports, not only `missing` ones.
 - Add real GPS-based camera location from the phone's sensor feed.
 - Support multiple concurrent camera nodes with per-node status and detection attribution surfaced properly in the Cameras and Analytics pages (the data model already supports this via `camera_id`; the UI and `/system` endpoints currently assume one node).
 - Investigate model quantization or a larger/edge-optimized inference target (e.g., ONNX/TensorRT export) to reduce the GPU dependency for field deployment.
 - Add browser-level (e.g., Playwright) tests for the frontend to close the gap identified in §9.2.
-

10. Appendices

Appendix 1 — Domain Terms

- **ALPR (Automatic License Plate Recognition):** the combined task of locating a license plate in an image (detection) and reading its characters (OCR).
- **YOLO (You Only Look Once):** a family of single-stage object detectors that predict bounding boxes and classes in one forward pass, chosen here (YOLOv8n) for its speed/accuracy balance on consumer GPU hardware.
- **mAP (mean Average Precision):** a standard detection-quality metric averaging precision across recall levels and (for mAP50-95) across IoU thresholds from 0.5 to 0.95.
- **JWT (JSON Web Token):** a signed, self-contained token used here to carry a user's id and role between requests without server-side session state.
- **WebSocket:** a persistent, bidirectional connection used here for the backend to push detection alerts to connected clients without polling.
- **Report lifecycle:** `under_review` (filed, not yet actionable) → `missing` (activated, eligible for plate matching) → `found` (recovered).
- **Camera node:** the standalone `vrrs-node` process; not a hardware device itself, but the software that turns a phone's video stream into detection events.

Appendix 2 — Installation

2.1 Backend

```
cd vrrs-backend
pip install -r requirements.txt
cp .env.example .env
# edit .env: DATABASE_URL, SECRET_KEY, CORS_ORIGINS
uvicorn app.main:app --reload
```

Runs at `http://localhost:8000`; interactive API docs at `/docs`.

2.2 Frontend

```
cd vrrs-frontend
npm install
cp .env.example .env
```

```
# edit .env: VITE_API_BASE_URL, VITE_WS_URL
npm run dev
```

Runs at `http://localhost:5173`.

2.3 Camera node

Requires a Python environment with `torch`, `ultralytics`, `easyocr`, `opencv-python-headless`, `requests`, `python-dotenv` installed (GPU-enabled `torch` strongly recommended for real-time performance).

```
cd vrrs-node
cp .env.example .env
# edit .env: MODEL_PATH (trained best.pt), PHONE_STREAM_URL, BACKEND_URL, CAMERA_
python plate_node.py
```

Requires an Android phone running an IP Webcam app on the same network/hotspot as the machine running `plate_node.py`.

2.4 Running everything together (development)

```
./run-all.ps1
```

Opens the backend, frontend, and camera node each in their own window.

2.5 Running the automated test suite

```
cd vrrs-backend && pip install pytest && python -m pytest tests/ -v
cd ../vrrs-node && python -m pytest tests/ -v
```

The backend suite uses an isolated in-memory SQLite database and does not touch the configured Postgres database. The node suite has no dependency on `torch` / `cv2` / `easyocr` — it only tests `plate_utils.py`.

Appendix 3 — Selected Source Code

3.1 `check-plate` matching endpoint (`vrrs-backend/app/routers/alerts.py`)

```

@router.post("/check-plate")
async def check_plate(data: AlertCreate, db: Session = Depends(get_db)):
    try:
        normalized = re.sub(r"^[A-Z0-9]", "", data.license_plate.upper())
        report = db.query(Report).filter(
            func.regexp_replace(func.upper(Report.license_plate), r"^[A-Z0-9]", "
            Report.status == "missing",
        ).first()
        if not report:
            return {"status": "CLEAR"}
        new_alert = Alert(report_id=report.id, location_spotted=data.location_spotted,
            camera_id=data.camera_id, confidence_score=data.confidence_score, image_id=data.image_id)
        db.add(new_alert); db.commit(); db.refresh(new_alert)
        await manager.broadcast({"type": "STOLEN_DETECTED", "alert_id": new_alert.id,
            "plate": data.license_plate.upper(), "location": data.location_spotted,
            "camera_id": data.camera_id, "confidence": data.confidence_score,
            "vehicle_make": report.vehicle_make, "vehicle_model": report.vehicle_model,
            "vehicle_color": report.vehicle_color, "detected_at": str(new_alert.created_at)})
        return {"status": "STOLEN", "alert_id": new_alert.id,
            "vehicle": {"make": report.vehicle_make, "model": report.vehicle_model}}
    except Exception as e:
        raise HTTPException(status_code=500, detail=str(e))

```

3.2 Fuzzy dedup logic (vrrs-node/plate_utils.py , written to fix §8.5's bug 2)

```

def is_recent_duplicate(recent_sightings, plate, now, cooldown_seconds,
    similarity_threshold=DEDUP_SIMILARITY_THRESHOLD):
    recent_sightings[:] = [
        (p, t) for p, t in recent_sightings if now - t <= cooldown_seconds
    ]
    return any(
        difflib.SequenceMatcher(None, plate, p).ratio() >= similarity_threshold
        for p, _ in recent_sightings
    )

```

3.3 Confidence normalization (vrrs-backend/app/routers/analytics.py , §8.5's bug 3)

```

NORMALIZED_CONFIDENCE = case(
    (Alert.confidence_score <= 1, Alert.confidence_score * 100),
    else_=Alert.confidence_score,
)

```

(Additional selected modules — `plate_node.py`'s detection loop, the React `AlertsContext`, and the full test suite — are included by reference to their file paths in the repository rather than reproduced in full here, per the golf thesis's own precedent of excerpting rather than reproducing entire files.)

11. References

- [1] Ultralytics, "YOLOv8 Documentation." Available at: <https://docs.ultralytics.com>. (Accessed during development, 2026.)
- [2] JaidevAI, "EasyOCR." Available at: <https://github.com/JaidevAI/EasyOCR>. (Accessed during development, 2026.)
- [3] Roboflow, "Roboflow — Computer Vision Dataset Management and Training." Available at: <https://roboflow.com>. Dataset export used: "zambia-number-plate-detection," v3, workspace `nicholass-workspace-j3yig`.
- [4] FastAPI, "FastAPI Documentation." Available at: <https://fastapi.tiangolo.com>.
- [5] SQLAlchemy, "SQLAlchemy Documentation." Available at: <https://www.sqlalchemy.org>.
- [6] PostgreSQL Global Development Group, "PostgreSQL Documentation." Available at: <https://www.postgresql.org/docs/>.
- [7] React, "React Documentation." Available at: <https://react.dev>.
- [8] OpenALPR, "OpenALPR — Open Source Automatic License Plate Recognition." Available at: <https://github.com/openalpr/openalpr>.
- [9] M. Jones, "JSON Web Token (JWT)," RFC 7519, Internet Engineering Task Force, 2015. Available at: <https://datatracker.ietf.org/doc/html/rfc7519>.
- [10] G. Bradski, "The OpenCV Library," Dr. Dobb's Journal of Software Tools, 2000. Available at: <https://opencv.org>.
- [11] pytest, "pytest Documentation." Available at: <https://docs.pytest.org>.
- [12] I. Sommerville, "Software Engineering," 9th ed., 2011. (Iterative development methodology reference, consistent with the phased approach in Chapter 3.)

[Add any interview sources, Zambia Police Service correspondence, or supervisor meeting notes you drew on for the requirements in Chapter 3 — those are real sources this environment has no access to and cannot cite on your behalf.]